Fakultät Informatik

Paper title

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Florian Meißner

Matrikelnummer: 3210376

Erstgutachter: Prof. Dr. Hans Löhr
Zweitgutachter: Prof. Dr. Michael Zapf

Inhaltsverzeichnis

Kapitel 1
Introduction

- Rust is increasing usage in system level
- but still many big projects (linux etc.) are written in C

Since the beginning of computer programming, there has been a discrepancy between the input states an interface formally accepts, and the input states that are sound to handle. For example, a reciprocal function $f(x) = 1/x$ might formally accept a 32 bit integer — and therefore all of its $2 \hat{} 32$ input states —, but the mathematical formula it tries to model will not give a sensible result for $x = 0$; least of all if the function in turn returns an integer, since there is no integer n: $1 / x = n$.

One straightforward solution has always been to limit the function domain via documentation. Users of that function are expected to read that documentation and recognize that it is a violation of interface contract to call it with $x = 0$. Violation of that contract would in turn result in an error, a crash, or — worse yet — Undefined Behaviour. An approach with drawbacks, as there are now two sources of truth about the function domain. One of these — the function signature, expressed in code — is already technically incorrect, as we have not stated a way to express "integers without zero" as a valid type (Listing 1). Additionally, as the program developes, the chance of both sources of truth to get further out-of-sync increases. For example, special behavior could be introduced to handle the undefined case by printing out errors. This discrepancy, between the high-level contract, expressible only in additional information in text form, and the function signature the compiler handles, raises the following question: Can we encode this precondition in such a way that the function signature makes it impossible to pass in values that violate any API contract?

```c
// `n` CANNOT BE ZERO!
double reciprocal(int n) {
  if (n == 0) {
    crash(); // we warned you!
  }
  return (double) 1 / n;
}
```

Listing 1: A typical C implementation of `reciprocal()`

In programming languages where we have strict and strong typing , we can enforce invariants about types we create, since we have to explicitly provide the methods of construction for values of these types, and if any invariants are not upheld, we can detect this unsoundness and trigger an error. This allows us to express function signatures of the kind discussed previously, by creating a new type `NonZeroI32` that represents the idea of an integer that cannot be zero. Unfortunately, not all languages provide the features necessary to formulate such powerful types. One prominent example is C, which is predominantly used in systems level programming, both in general and in the domain we are looking at in this work. Besides the basic datatypes that exist primarily to differentiate between CPU directives — integers, floating points, characters, pointers — users can create composites of these types via the `struct` or `union` constructs, and define functions to work on these datatypes only. But these type requirements can easily be subverted, because in C, casting between different types is often implicit, and there are no mechanism to enforce invariants of a type — all user code that can "see" a struct can create or delete instances as it sees fit.

That's why, in this work, we will look at Rust: a modern language that is gaining rapid traction in the area of systems programming, and has a strong, expressible type system as one of its flagship features. This enables us to explore the latter approach. Listing 2 shows such an implementation in Rust. It utilizes the concept of a "newtype struct": a `struct` with one anonymous member, that is used to wrap this inner element and to effectively give new type semantics to it. If the inner value is chosen to be private, ergo not visible or accessible from code from a different module, this prevents any alternative way of constructing or modifying values of this new type other than what the module creator chooses to provide. This achieves exactly the desired effect. By making the inner value private, we then ensure that users of our library are forced to use the only construction method we provide them with, `NonZeroI32::new(i32)`. This `new()` function can be total, since it returns a result value representing a fallible computation. In that sense, the function signature of `new()` expresses that **every 32-bit integer is either a valid non-zero 32-bit integer or an error**.

```rust
pub struct NonZeroI32(i32);

impl NonZeroI32 {
  pub fn new(n: i32) -> Result<Self> {
    if n == 0 {
      Err("n == 0 is not allowed!")
    } else {
      Ok(n)
    }
  }
}

pub fn reciprocal(n: NonZeroI32) -> f64 {
  // ...
}
```

Listing 2: A new type `NonZeroI32` that represents the idea of a 32-bit integer **guaranteed** to not be zero

This is one of several features of Rust that promise improving soundness checking and language expressibility with no or minimal runtime impact. System programming is an area where these soundness guarantees are especially important. While a logic error stemming from an unchecked invariant in an application can crash this application or corrupt its data, a kernel or Operating System (OS) bug can cause those failure states in any and every subsystem and program. Faulty OS behaviour can cause instability on every layer of the system, and endanger the work of all users. Simultaneously, system programming has to produce performant instructions, since they will be running as the backbone of the OS, which often leads to complex data structures where keeping invariants as cognitive load, for programmers to check upon every code change, is unlikely to produce said stability in the long run. That is also why solutions that penalize compile time only are especially valuable, because compile time is often expendable — after all, most OSs run many times as often as they compile.

Our work is therefore targeting a subsystem of modern OS development. Filesystems were chosen, because they fulfil all stated criteria: their performance matters, their correctness is crucial for stable system usage, and they deal with complicated, highly optimized data structures where invariants between those structures play a major role. Kernel module development is not a trivial task, though, and challenges during code writing and testing would cost additional time. Debugging is also comparably harder when the targeted module is injected into the OS itself. FUSE (**F**ilesystem in **USE**rspace) offers a way out for this conundrum. It is a framework that works by combining a kernel module with a userspace library. Filesystems using it link against the userspace library and use it to communicate with the kernel module, which will redirect OS requests to the userspace program. This provides an environment not unlike a sandbox: the kernel module is well known and tested, and approximately bug-free. The concrete filesystem, which we will implement, will live in userspace and therefore be easy to debug and not critical to system

stability. And since the APIs are quite similar, approaches found to be working when modeling FUSE filesystems will with high probability also work in general, e.g. as native kernel modules.

[3]

Kapitel 2
Background

2.1 Rust

2.2 FUSE

Kapitel 3
Related work

3.1 `rust-fatfs`[1]

A Rust reimplementation of the FAT filesystem standard created by Microsoft. It is implemented as a kernel module, whereas our library lives in userspace with the usage of FUSE. Although the authors claim exploring security and safety benefits as motivation, the evaluation focuses only on performance, benchmarking the work against the established C kernel module. We did not consider empirical performance analysis, and focus on which kinds of CVEs can be prevented.

3.2 `fuser`

The `fuser` crate, maintained by GitHub user `cberner`, is currently the most used FUSE bindings crate on `crates.rs`, counting over 150k downloads per month, and over 1k stars on GitHub. It is not an academic project, but a practical solution for integrating FUSE into the Rust ecosystem, thereby leveraging the advantages of Rusts architecture for filesystem development. While our goals are not conceptually different as a whole, there is no explicit focus on security. Types are modeled very closely to the underlying C architecture, although users aren't forced to create C ABI compatible functions, since that — like in our solution — is abstracted away. In fact, probably due to increased performance, `fuser` does not use the `libfuse` C library (`libfuse`) but talks directly to the FUSE module via socket. This is a complex, low-level undertaking and was out of scope for our project, although it would have been interesting in principle, as the same design principles could still be applied. Still, results are comparable because the crate models `fuse_operations` very closely to `libfuse`. Also, while we chose to use the high-level `libfuse` API, which identifies files via paths directly and submits results synchronously, `fuser` leverages the low-level (LL) `libfuse` API, which uses Inodes to represent entries and message helper structs to return asynchronously. This was carefully considered, but the high-level API simplifies some implementation parts while losing almost none of the targeted design space for safe systems programming, so it was deemed the better choice.

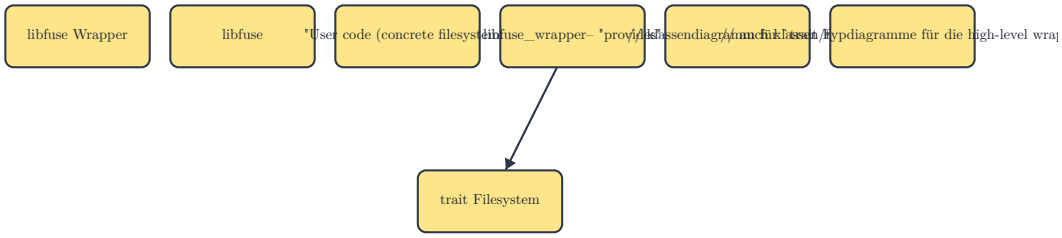
[4]

3.3 Rust for Linux[2]

Kapitel 4
Concept

[5], [6], [7], [8], [9]

Kapitel 5
Implementation



5.1 Basic C interop

Safe Rust can never (sans compiler errors) cause UB in the resulting binary program. In unsafe Rust, this is not the case; the programmer now has to uphold several invariants to ensure soundness. In the C standard, where behavior in any situation not explicitly defined by the language standard is implicitly "undefined", Rust limits these invariants to a set of specific, well-documented cases. This makes reviewing the soundness property of unsafe code easier.

5.1.1 Pointers

Regarding use of raw pointers in unsafe Rust, the following invariants exist:

1. No dereferencing of dangling or unaligned pointers.
2. Respect aliasing rules: no pointer is allowed to point to memory that's also pointed-to by a mutable reference, since a mutable reference in Rust is guaranteed to be exclusive.
3. Respect immutability: no pointer is allowed to modify data that's also pointed-to by a shared reference, since a value behind a shared reference is guaranteed not to change.
4. Values in memory must be valid for their respective types: pointers must not be used to change the representation in memory of to a value — or reference — to a state which is not valid for the type this value — or reference — has. E.g. a `NonZeroU8`, represented in memory

as a `u8`, will have one bit pattern that would correspond to a numeric zero and is therefore illegal.

Because `libfuse` calls all our callbacks with at least one C pointer, we have to check these invariants as rigorously as possible before we call into user code, if we want to eliminate them as sources of UB.

1. We have to differentiate between three cases:
   - **Unaligned pointer**: this is easy, as Rust provides `ptr::is_aligned()`, which takes a pointer and detects misalignment.
   - **Dangling null-pointer**: this is also easy, both manually and through the Rust-provided `ptr::is_null()`, which takes a pointer and detects null-ness.
   - **Dangling non-null pointer**: this happens when a pointer is used-after-free or if pointer arithmetic goes wrong, and is much harder to avoid. Since we don't control memory allocation in C, we largely have to trust C code to not pass us pointers from this category. This would cause UB and should therefore be documented visibly as soundness assumption.
2. For pointers passed to us by `libfuse`, the solution is simply to not create a reference to it. If it is necessary to pass a mutable reference into user code, an intermediate owned value must be created, and the target value must be copied in and out of that intermediate.
3. When dealing with non-const pointers, care must be taken to not create a shared reference to it. Const pointers don't matter for that aspect, since it is impossible to modify values through them, given they are not cast to non-const pointers.
4. This only matters when primitive C-style casts or `mem::transmute()` are used, as otherwise the Rust typesystem protects us from writing values of the wrong type, even inside unsafe blocks. Writing to a pointer can involve writing raw bytes; if that is required, extra care must be taken, and it is therefore usually better to avoid this.

5.1.2 Strings and Unicode

Rust's native string types (`str`, `String`) exclusively store UTF-8.[10, ch. 8.2] The main kind of strings this library needs to handle are the file paths that filesystem callbacks are called on. The encoding of those is platform-dependent, usually being C-like ASCII strings on Unix-like systems and UTF-16 on newer Windows versions. Correctly detecting and handling string encodings is a hard problem , and since UTF-8 is a superset of ASCII, we chose to not handle UTF-16 or other cases and emit an error when encountering non-UTF-8 input. This limits the complexity of the prototype without limiting the scope of the reseach question.

5.1.3 Unwinding across FFI boundaries

When a Rust program is compiled with stack unwinding support and a panic is triggered, the default uwind handler will walk up the stack in order to react to the panic, collecting debug information or cleaning up data. In a program using FFI, this can lead to crossing into another language runtime while walking the stack. Doing so correctly is a non-trivial task and can easily lead to UB.[11, ch. 14] On the other hand, turning unwinding off loses helpful stack traces and debug information when a panic happens. We therefore decided to keep unwinding behaviour while preventing any panic from propagating across an FFI boundary.

Every function that is visible to C can potentially be called from an environment where unwinding works differently or not at all. Therefore each of those functions must be panic-free. As of now, there is no compiler flag or lint that detects or prevents use of panicking functions, operators or language keywords. As a result, this must be done manually by reviewing the source code of the functions in question, and, recursively, the functions they call. A convention exists to note possible panics in a section of the function documentation, but even the standard library doesn't consistenly follow it.

One crate that tackles this problem is `no_panic`[1] by David Tolnay, a prominent figure amongst the Rust community. It provides the ability to annotate function declarations with an attribute macro, and promises to halt the compilation with an error if the function is **not provably panic-free**. This implies that it is possible to write functions that would not panic, but would still not compile if the compiler is unable to prove that property. The crate thereby takes a stance typical of Rust philosophy: it is preferable to reject sound programs, than to accept unsound ones.

While preventing panics in self-maintained code requires careful manual analysis, this is not possible for user-provided functions. For this, there exists a function `panic::catch_unwind()`[12] that takes a closure and executes it, catching any unwind that would occur and returning an error instead. Wrapping the call to user code inside this function ensures that no panic will be propagated up the call stack from this point on.Listing 3

[1] https://docs.rs/no-panic/latest/no_panic/

```rust
fn call_into_user_code<FS: Filesystem, T>(
    method: &str,
    user_fn: impl FnOnce() -> Result<T, Errno>,
) -> Result<T, (String, Errno)> {
    let fs = std::any::type_name::<FS>();
    std::panic::catch_unwind(core::panic::AssertUnwindSafe(user_fn))
        .map_err(|panic| {
            // abort, since internal state of filesystem impl can now be inconsistent
            state::clear();
            (
                format!("PANIC on `{fs}::{method}`:\n\n{panic:?}\n"),
                Errno::ENOTRECOVERABLE,
            )
        })
        .and_then(|inner| inner.map_err(|e| (format!("Error in user code `{fs}::{method}
`"), e)))
}
```

Listing 3: An exemplary trampoline function signature implementing compile-time static dispatch via generics

5.2 FUSE operations

The `libfuse` operations struct contains 43 callbacks to implement, most of which are optional and not needed for a filesystem to work properly. If we can forgo modifying the state, and create a read-only filesystem, the number of required calls can be brought down to 3. [13, p. structfuse___operations.html], [13, p. example_2hello_8c.html]

Some of the operations which are superflous for our experiments include:

- `lock`/`flock`: These are used for file locking, enabling safe concurrent access, and locking primitives across processes. Since our filesystem is readonly, no locking is needed.
- `ioctl`: Needed for special I/O commands, when simple seeking to byte offsets, and reading/writing from them is not sufficient. Examples include ejecting CD-ROM or rewinding data tape. Not needed for our general-purpose minimal filesystem.
- `write`/`sync`/`fsync`: These are used for writing data out, and to force flushing buffers to the underlying storage. Irrelevant in a read-only filesystem.
- `mkdir`/`link`/`create`/`mknod`/`unlink`/`rmdir`: Creating and deleting of entries of various types. Not relevant for a read-only filesystem.

All implemented operations check their pointer arguments for validity, with the methods discussed earlier (Section 5.1.1). The obligatory `path` argument, that identifies the entry to operate on, is converted from a C string into native Rust, and also checked for validity (Section 5.1.2). After basic correctness of inputs has been ensured, the code tries to load the filesystem object from the global registry. This is done to minimize unneccessary work when some inputs are not sound. The

load could fail, e.g. in case the user code triggered a panic earlier, or due to a bug in the wrapper library. This is also handled (Section 5.3).

After that, some operation-specific instructions are executed, and a context is set up to call into user code without triggering panic unwinding (Section 5.1.3). `libfuse` datatypes are converted to our Rust representations, adding the implicit safety checks. After the user code has been executed, the results are converted back into `libfuse` types, and success of the operation is signaled up the stacks.

Next up is a detailed description of the required calls, accompanied with their respective C and Rust signatures.

5.2.1 getattr

```
int(*   getattr )(const char *, struct stat *, struct fuse_file_info *fi)
```

```
pub unsafe extern "C" fn getattr<FS: Filesystem>(
    path: *const i8,
    stat_out: *mut libfuse::stat,
    _fuse_file_info_out: *mut libfuse::fuse_file_info,
) -> i32
```

This provides information about a filesystem entry, be it a file, a directory, a symbolic link, or some sort of special device. Without this call, no metadata could be queried about our filesystem, and its usefulness would be severely limited.

This function, as do all of the callbacks described here, takes a path in form of a C string to describe the filesystem object to query. It also takes two additional parameters: a `stat` struct as output, to be filled with the resulting metadata (Section 5.4.2), and an optional `fuse_file_info`, which may be set when the file in question is currently opened, and can provide additional metadata and FUSE settings if set.

We chose to mostly ignore the `fuse_file_info` for now, as it is only used under specific circumstances, and even then provides only very specialized attributes that would go beyond the scope of this exploration. The other two parameters are checked as per concept.

It's the users job to create an instance of `struct Stat` and pass it back to us. This newtype struct contains a valid `stat` fuse struct inside, which is needed as return value written into the output pointer argument of same name, and can trivially convert to one.

5.2.2 readdir

This function's job is, given a directory as path, provide a list of child entries, enabling directory content listing (as e.g. in the `ls` command). An offset can be provided to support partial listings

over multiple calls, however we chose to ignore this, as is allowed in the documentation.[13, p. structfuse___operations.html]

There exists an alternative, more complex mode, which we could have chosen to support: Implement the additional `opendir` operation to open the directory to enumerate as a file descriptor. Then, `readdir` is called on the active file descriptor, providing a view of the directory that is guaranteed to be the same as when `opendir` was called. This was deemed unneccessary to explore the given research questions, and skipped subsequently.

The API is designed, so that the `readdir` implementation doesn't just return, or write into, an array of entries. Instead, a function pointer to a "filler" function is provided. Our operation has to call this function for every entry. Some of the filler functions parameters correspond to entry metadata, others, like a pointer to an opaque data buffer, have to be forwarded.Listing 4[13, p. structfuse___operations.html]

```
// ...
for entry in entries {
    let entry_as_c_string = try_errno!(CString::new(entry.clone()).map_err(|e| {
        (
            format!("converting dir entry '{entry}' into a C string: {e:#}"),
            Errno::EIO,
        )
    }));
    debug!(?path, "filling entry '{entry}'");
    let fill_result = unsafe {
        filler_fn(
            data_ptr,
            entry_as_c_string.as_ptr(),
            ptr::null(), /* setting `stat` struct to NULL, as per `hello.c` */
            0,           /*: offset */
            libfuse::fuse_fill_dir_flags_FUSE_FILL_DIR_DEFAULTS,
        )
    };

    if fill_result != 0 {
        bail_errno!(
            format!("filler_fn returned non-zero for '{entry}': {fill_result}"),
            Errno::EIO
        );
    }
}
// ...
```

Listing 4: Excerpt from the `readdir` trampoline, passing the Rust vector of directory entries into the C filler function.

5.2.3 read

The `read` operation provides us with the means to fetch the content of files, complementing our set of operations to obtain a usable, if minimal, filesystem. It takes as parameters a size and an offset, determining the range of content to be read, as well as a C character array to store the data in. Again, an optional `fuse_file_info` struct pointer is provided, which we can ignore.

Dealing with the size and offset parameters provided a challenge. While the requested size parameter is of type `size_t`, which is usually defined as 64-bit integer on modern systems to be used for indexing memory, we have to return the actual amount of read bytes as signed 32-bit integer. As such, we can only signal successful reads of up to 2^31 - 1 bytes. Further investigation showed that maximum read size is limited by the Linux kernel[14], so the limitation in the API seems reasonable, assuming other operating systems impose similar limits. We therefore have to carefully check if the input parameter is inside the allowed range, and convert between the respective integer types, in addition to the usual checks. Furthermore, Rust does not provide a native method of specifying an upper bound for a vector length as part of the type signature, so manual checks are required after the user code returns the data. Dependent types or refinement types would be a possible solution, but are not available in Rust aside from research projects.

If the size checks pass, a pointer copy is issued, for which Rust STD provides a function. Because we use `unsafe`, we documented the assumptions made and invariants we checked, as is common practice.Listing 5

```
  // Safety: we checked that the buffer is big enough to hold the returned data (if
`size` argument was correct).
  //         Also we checked that the pointer is aligned and non-null.
  //         Also, the areas cannot overlap, since the Rust vector has reserved its own
memory.
  unsafe {
      ptr::copy_nonoverlapping(
          result.content.as_ptr(),
          buf as *mut u8,
          result.content.len(),
      );
  }
```

Listing 5: Excerpt from the `read` trampoline, copying the read result into the provided C buffer.

5.3 Initialization and Global State Management

- We need to supply a number of C functions that know which user impl to call
- Possibility: just use data pointer from `libfuse::init`
  - Con: push raw pointers around, prone to corruption
- My choice: use generics, overload generic trampolines with user Filesystem type

- that way, the compiler generates a concrete version (with its own address and hard-coded user code address) of our generic trampolines
- since generic parameter is the only type with `impl Filesystem` in scope, type system prevents any confusion/programmer error.
- since compiler generates hardcoded version, memory corruption due to logic errors anywhere is also not a problem
- con: only one instance per struct type per process.
  - workaround: just use wrapper structs (can be done easily from user code)

Since the libfuse initialization routine takes a struct of callback function pointers (`fuse_ops`), that creates the following problem. Since the C signature is predetermined, user functions cannot be used, because that would force signatures of user functions to use the lower-level C types which we try to avoid. That means, even though there is a one-to-one correspondence between FUSE operation callbacks and trait methods on the `Filesystem` trait, they are not compatible and cannot be used interchangably. The obvious approach is to provide **trampoline functions (trampoline_functions)**, which then wrap, transform and safety-check the C type values on call and dispatch into user code. A non-trivial problem, one that is not obvious at first sight, is how the trampoline knows which filesystem implementation to dispatch to. There are two basic options how to use the trampolines:

1. Use one global trampoline per callback, and somehow transport the choice on which filesystem to use inside the C arguments that **our `libfuse` wrapper** gets passed by **`libfuse`**.
2. Somehow generate a set of trampolines per user filesystem, which are then hard-coded towards the specific filesystem implementation.

A way to implement option 1 is provided in the form of a `void *private_data` pointer that can be passed to **`libfuse`** during filesystem registration. This pointer can contain arbitrary user-specified data, and is not used by **`libfuse`** except for making it available to every fuse operation via the `fuse_get_context`[2] function.

Since it is possible to store a Rust pointer inside a C void pointer, **our `libfuse` wrapper** can submit a pointer to the user implementation as payload for `private_data`, then let each trampoline poll the FUSE context struct, cast the void pointer back to a trait object reference and dispatch into the corresponding trait method. This has the following disadvantages:

- Decaying a managed Rust reference into a raw pointer loses the advantage of lifetime tracking that is one of Rusts fortes in the undertaking of creating safe systems-level code. Manual care has to be taken not to invoke a use-after-free, accessing an uninitialized or unauthorized memory location or — in the best case — simply leaking memory. In fact, the safest option would be to initialize this data pointer once, and then never free it, since it is the dealloc part that introduces memory unsafety to a system , and even if leaking memory (and not calling

[2]https://libfuse.github.io/doxygen/fuse_8h.html#a5fce94a5343884568736b6e0e2855b0e

destructors) is acceptable, since `libfuse` passes around non-const pointers to everything, bugs at any point of both our trampolines and `libfuse` can easily lead to access of corrupted pointers and therefore to UB. This is usually a tradeoff that must be accepted when dealing with FFI into unsafe languages, but should be mitigated whenever feasible.

Both disadvantages would in theory prevented by a solution after option 2, and thankfully, with the use of generics, Rust brings includes the tools to implement such a solution. As seen in Listing 6, this exemplary trampoline function is generic over types implementing our `Filesystem` trait. This leads the Rust compiler to generate a concrete, independent `getattr` trampoline function for every trait implementation of `Filesystem` that is used to call our initialization function. The generic approach is then combined with a singleton registry[3] which provides a global map of values, indexed by types. We can now store the concrete user-supplied filesystem struct inside this registry and use the type of this filesystem struct as index, which additionally will be deduced implicitly by the compiler from the argument types of our initialization function. That means, given there are no other implemenations of our `Filesystem` trait in scope when declaring the generic functions, the type system guarantees us that the user's type is the only one that can be used for dispatching, shielding even against potential programmer oversight.

This has the drawback of only allowing one instance of a concrete `Filesystem` type to be mounted per process. But since — if needed — newtype structs can be used to create different concrete types with minimal boilerplate, this was deemed tolerable.

```
pub unsafe extern "C" fn getattr<FS: Filesystem>(
    path: *const i8,
    stat_out: *mut libfuse::stat,
    _fuse_file_info_out: *mut libfuse::fuse_file_info,
) -> i32 {
```

Listing 6: An exemplary trampoline function signature implementing compile-time static dispatch via generics

5.4 Type modeling

Creating thin high-level representations of the low-level data types that make up the `libfuse` API, that nonetheless verify as many correctness properties as possible, is the main focus of this project. Where feasible, these properties are checked during compile-time, which gives the additional advantage of not impacting runtime performance. Otherwise, runtime checks are emitted to still provide correctness, but at the disadvantage of producing runtime errors instead of halting compilation, which increases development cost. It would be common practice in low-level Rust crates to provide `*_unchecked()` variants for these runtime-checked methods, to give users the

[3] https://crates.io/crates/singleton-registry

choice of circumventing those checks and trading performance for possible UB. Due to the goals of this work, and time constraints, this was mostly skipped.

5.4.1 Typed builder

A pattern that is often encountered in Rust is a *builder*. It tries to solve the problem of value creation, where, for big types, many values must be provided, some of which are interdependant or introduce combined constraints, and some are only available at different times.

There are multiple ways to construct a value in Rust:

- **Initializing a raw struct**: If the struct has only public fields, directly constructing it is possible. As downsides, validation of the value as a whole is not possible, since a struct can always be legally constructed from legal values of all its elements, so any checks must be performed through the types of its members. Also, staggered initialization is not possible, since at construction point, every value has to be provided. It is possible to let construction use default values for some members, but this doesn't equal partial initialization, since it is not fundamentally possible to tell an uninitialized value from a valid value equalling the default value of the field. Thus, this loses some type safety.
- **Constructor function**: Rust doesn't have constructors as language constructs, as opposed to e.g. C++. Idiomatically, constructors are member functions with the name `new()` or `new_*()`, since Rust also doesn't allow function overloading. Paired with lack of default parameter values, this makes writing constructors rather rigid, and is not substantially different to using raw structs. No staggered initialization is possible, as with a struct initialization, but interdependent validity checks can be performed; with the caveat, that when a type provides multiple constructors, all of them must duplicate the verification logic or use a private shared constructor that centralizes the checks. This is also manual, and can be overlooked, and the probability of oversight increases with the count of constructors.
- **Struct as constructor parameter**: An elegant combination of the two concepts above is to use a dedicated initialization struct, that is either passed to a constructor or can be cast to the target type. It acts as a sort of dictionary, or list of named parameters. This emulates named arguments and also works with private fields, since the target type construction is hidden from the user.I It still doesn't solve staggered construction, for the same reasons as stated above. But one could maybe imagine using this pattern multiple times, where each initialization struct sets some parameters and generates the next stage of initialization via an opaque intermediate type, thus providing multiple phases of initialization. Implementing this from hand would be complex and error-prone, since every combination of initialized-ness has to be coded explicitly, which leads to $n!$ cases, where n equals the number of initialization parameters.
- **Typestate builder pattern**: This is where Rusts strengths come in to play. Rust provides a rich macro system, enabling the generation of the boilerplate code that is needed for such

a multi-stage “build” of a value. The pattern described here is named “typestate builder” pattern, or “typed builder” pattern, and is one variant of the more general builder pattern. It annotates the target struct, generating a builder struct which allows to set every parameter by itself. Required parameters must be set exactly once, optional parameters zero or one times. This detects possible bugs as cases, where a value is forgotten to be set, or is set too many times, overwriting the previous choice. Additional checks can be added to every member and the struct as a whole, in the form of closure predicates. Syntactic sugar for flags is supported to be able to write `boolean_flag()` to enable a flag, and omission of any call to leave it of, instead of `set_boolean_flag(val: bool)`. This improves readability and provides additional correctness checks. Also, only the validity closures live in runtime; the basic correctness checks of every field being set the right amount of times are modeled within the type system, leading to compilation errors when disregarded, which is one of our goals.

The implementation of this builder pattern creates a utility intermediate type for every possible combination of whether a type has been set. An example would be a builder for a struct `struct Point { x: f32, y: f32 }`, which represents a point on a cartesian coordinate system. A manual implementation of a builder could look like this:

```
struct PointWithXSet { x: f32 }
struct PointWithYSet { y: f32 }
struct PointWithNothingSet {}

impl PointWithNothingSet {
  pub fn set_x(x: f32) -> PointWithXSet { /* ... */ }
  pub fn set_y(y: f32) -> PointWithYSet { /* ... */ }
}

impl PointWithXSet {
  pub fn set_y(y: f32) -> Point { /* ... */ }
}

impl PointWithYSet {
  pub fn set_x(x: f32) -> Point { /* ... */ }
}

impl Point {
  pub fn build() -> PointWithNothingSet { /* ... */ }
}
```

Listing 7: FIXME.

In this case, the type system makes it invalid to set an x or y coordinate multiple times, or forget to set it. There are only two possible ways to obtain a value of type `Point`:

- calling `Point::build().set_x().set_y()`
- calling `Point::build().set_y().set_x()`

Therefore we can ensure proper initialization of the point value.

Another way of modeling this pattern uses generics. Listing 8 shows such an approach.

```
pub struct PointBuilder<State: PointBuilderState> { state: State };

trait PointBuilderState {}
struct NothingSet;
impl PointBuilderState for NothingSet {};
struct XSet { x: f32 };
impl PointBuilderState for XSet {};
struct YSet { y: f32 };
impl PointBuilderState for YSet {};

impl Point {
  pub fn build() -> PointBuilder<NothingSet> { /* ... */ }
}

impl PointBuilder<NothingSet> {
  pub fn set_x(x: f32) -> PointBuilder<XSet> { /* ... */ }
  pub fn set_y(y: f32) -> PointBuilder<YSet> { /* ... */ }
}

impl PointBuilder<XSet> {
  pub fn set_y(y: f32) -> Point { /* ... */ }
}

impl PointBuilder<YSet> {
  pub fn set_x(x: f32) -> Point { /* ... */ }
}
```

Listing 8: FIXME.

On closer inspection this pattern bears resemblance of a state machine, where states are marker structs — structs with no associated data fields — that implement a marker trate — analogously, a trait without associated items —. Methods on these concrete types, which after monomorphisation are the `PointBuilder` with a concrete state as type parameter, that return a `PointBuilder` with a different state type, represent transitions between those states. This is intuitive because, as a transition can only be applied to the start state and results in the end state of that transition, methods on a type can only be run on an existing value of that type, and always produce the return value.

The implementation using generics has a few advantages: Since all intermediate types are specializations of a general builder type, there can be methods on the general builder type, which correspond to transitions on any starting state.

While a typestate builder has many advantages in statical correctness, conditional branches can be difficult to handle. That is because every state of the builder is effectively a different type, and Rust doesn't allow items to have different types depending on a branch. This is encountered frequently when dealing with complex generics, and while staying inside this type abstraction, there is no solution besides avoiding conditionally setting fields, and instead executing the conditional code only while calculating the value . Runtime builders don't have this issue, as every builder state has the same type, and the information of which field has been initialized is usually stored through optional types.

Because of the tradeoffs discussed between the different solutions, it can be advantageous to provide the user with multiple approaches, enabling them to choose whatever tool appropriate for the context. For example, we chose to provide both a type builder and a runtime builder for our `FileMode` struct, allowing correctness when flow of execution is well-known, and flexibility with runtime checks, when it's not.

[15]

5.4.2 `stat`

The `libfuse` `stat` struct is very similar to the namesake found in the POSIX standard (POSIX). Both describe an entry in an abstract filesystem, and contain most of its attributes. This set of attributes is needed for most interaction, because it provides data not limited to: the type of the entry — file, directory, symbolic link or other — it's permissions, size and modification dates. It is usually the set of information our wrapper has to provide to the surrounding system when some interaction with the filesystem takes place, e.g. listing or changing into a directory, or opening a file. [16]

Our attempt at modeling lead us to break down the struct into smaller parts, which require more attention:

- `FileType`, which is an enum flag of several possible values that have to specifically match magic IDs from the corresponding C header.
- `FilePermissions`, which are stored as a positive integer and usually displayed as an octal number in the range of `0o000` to `0o777` and represent restrictions on reading, writing and executing the underlying entry.
- three bitflags (`setuid`, `setgid`, and `vtx_flag`), that are context-dependent and enable additional features. These are stored inside the permissions integer in the underlying Unix APIs.

Other fields, like file size and modification time, were not deemed as interesting, since it can be correct for them to assume every valid bit pattern the underlying C type can represent, and checking the correctness semantically would introduce significant runtime overhead. E.g. validating modification time would have to detect modification in arbitrary files, and file size is an attribute that the wrapper has no insight into. Further insight into this problem is provided in Section 6.2.

5.4.3 `fuse_file_info`

This type, although a parameter to every operation implemented, is optional in every case, and seldomly used. In fact, due to the limited nature of our experiment, because we don't work with `open` and file descriptors, it can be assumed that the struct never be initialized. Therefore, modeling was skipped. This does not mean that `fuse_file_info` is not a good candidate for our methodology. To the contrary: because it contains a bitset with various flags, these can easily be modeled with associated methods, which atleast provide a simple safeguard against erroneous bit operations. Additionally, some entries influence further behaviour of the filesystem and could probably be used to implement more correctness checks.

5.4.4 `FileMode`

`FileMode` is an abstraction that `libfuse` provides, that encapsulates both file type and the permission mask. To stay close to the lower level, we opted to keep this encapsulation.

5.4.5 `OpenFlags`

5.5 Error handling

While `libfuse` and Rust both return an error value

Kapitel 6
Evaluation

To evaluate the effectiveness of our approach, we collected a sample of CVEs found in the filesystem modules of the Linux kernel in recent years. We then categorized them based on whether the approach discussed here would have been effective in preventing them. We also implemented a simple prototype filesystem using our wrapper, to provide a reference point for the ergonomy and expressiveness of the API.

6.1 CVEs

The CVE (Common Vulnerabilities and Exposures) system is an internationally accepted, de-facto standard, cataloguing system for vulnerabilities TODO

[17]

6.2 A prototype filesystem: `hello2`

To test our wrapper library, we created a minimal filesystem using it. It implements only three callbacks — `getattr`, `read`, `open` — as this is enough to provide a complete, usable filesystem.[13, p. example_2hello_8c.html]

This filesystem is read-only, since that narrows down the functionality we have to implement. Files are declared in a static global array, and are even associated with a closure object, to facilitate files with dynamic content. The following example (Listing 9) shows a global file table of two entries: `time.txt`, which always reads the current system date and time, and `pid.txt`, which always reads the ID of the filesystem process. This dynamic property of our test filesystem allows us increase confidence in our abstractions, by providing less stability on which to accidentally depend.

Additional logic was deemed necessary to be able to build a hierarchical recursive folder data structure from the provided file table, which is needed for listing directory contents. Implementing this keeps the file table itself clean and readable, and accelerates development and testing.

Besides some boilerplate to iterate over files in a folder, the only logic consists of the block `impl Filesystem for Hello2`, where we implement methods on our `libfuse` wrapper's filesystem trait. The implementations were straight-forward and simple, which which was one of our `libfuse` wrapper. Most low-level details and pitfalls were abstracted away. One aspect that required a proportionally high amount of SLoC was dealing with partial and offset reads, but offloading that to the wrapper would mean that the user has to provide an array with the full file content already contained, only for the wrapper to calculate the correct offsets. This could be made possible as an additional opt-in API, but would almost certainly result in major inefficiencies, as the filesystem has to procure the whole file's content every time a partial read is requested.

```
static FILES: LazyLock<[FileEntry; 6]> = LazyLock::new(|| {
    [
        ("/pid", Box::new(|| std::process::id().to_string())),
        (
            "/time",
            Box::new(|| format!("{}", chrono::Local::now().format("%c"))),
        ),
    // …
```

Listing 9: A global file table for our `hello2` example filesystem

Kapitel 7
Conclusion

7.1 Limitations

Kapitel 8
Future work

Bibliography

[1] S. Oikawa, "The Experience of Developing a FAT File System Module in the Rust Programming Language," in *Software Engineering, Artificial Intelligence, Networking and Parallel/ Distributed Computing*, R. Lee, Ed., Cham: Springer International Publishing, 2023, pp. 45–58. doi: 10.1007/978-3-031-19604-1_4.

[2] "Rust for Linux." Accessed: Feb. 18, 2026. [Online]. Available: https://rust-for-linux.com/

[3] W. Bugden and A. Alahmar, "Rust: The Programming Language for Safety and Performance." [Online]. Available: https://arxiv.org/abs/2206.05503

[4] C. Berner, "fuser: Filesystem in Userspace (FUSE) for Rust." [Online]. Available: https://github.com/cberner/fuser

[5] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "Safe systems programming in Rust: The promise and the challenge," *Communications of the ACM*, vol. 64, no. 4, pp. 144–152, 2020.

[6] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk, "System Programming in Rust: Beyond Safety," in *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, in HotOS '17. Whistler, BC, Canada: Association for Computing Machinery, 2017, pp. 156–161. doi: 10.1145/3102980.3103006.

[7] V. Astrauskas, P. Müller, F. Poli, and A. J. Summers, "Leveraging rust types for modular specification and verification," *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, Oct. 2019, doi: 10.1145/3360573.

[8] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers, "How do programmers use unsafe rust?," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020, doi: 10.1145/3428204.

[9] L. Seidel and J. Beier, "Bringing Rust to Safety-Critical Systems in Space," in *2024 Security for Space Systems (3S)*, 2024, pp. 1–8. doi: 10.23919/3S60530.2024.10592287.

[10] S. Klabnik, C. Nichols, C. Krycho, and Rust Community, "The Rust Programming Language." Accessed: Feb. 18, 2026. [Online]. Available: https://doc.rust-lang.org/book/

[11] The Rust Project Developers, "The Rust Reference." Accessed: Feb. 18, 2026. [Online]. Available: https://doc.rust-lang.org/1.92.0/reference/

[12] The Rust Project Developers, "The Rust Standard Library." Accessed: Feb. 18, 2026. [Online]. Available: https://doc.rust-lang.org/1.92.0/std/

[13] "libfuse API documentation." Accessed: Feb. 21, 2026. [Online]. Available: https://libfuse.github.io/doxygen/

[14] “read(2) — Linux manual page.” Sep. 21, 2025.

[15] I. Arye, “typed_builder: Compile-time type-checked builder derive.” [Online]. Available: https://github.com/idanarye/rust-typed-builder

[16] “stat(3type) — Linux manual page.” May 02, 2024.

[17] The MITRE Corporation, “2025 CWE Top 25 Most Dangerous Software Weaknesses.” Accessed: Feb. 13, 2026. [Online]. Available: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

Kapitel 9
Glossary

***OS* – Operating System** 3, 3, 3, 3, 3, 3, 3

***POSIX* – the POSIX standard** 23

Rust: TODO 11, 11

***UB* – Undefined Behaviour**: TODO 1, 11, 12, 13, 19, 20

`*libfuse*` – the `libfuse` C library 7, 7, 7, 7, 12, 12, 14, 15, 15, 18, 18, 18, 19, 19, 19, 23, 24, 24

our `libfuse` wrapper 18, 18, 26, 26

newtype struct: TODO 15, 19

soundness: TODO define & quote 11, 11

***trampoline_function* – trampoline function**: A wrapper function which has the job of being a proxy for another, binary-incompatible function, setting up the correct environment and converting the call. In our case, trampoline functions are ones that have the correct ABI and signature to be passed as FUSE operations, and, when executed, redirect execution flow into Rust code providing the actual filesystem functionality. 18